

Diseño de Arquitecturas Web Escalables

1. Escalabilidad en aplicaciones web

La escalabilidad es la capacidad de un sistema para manejar un aumento en la carga de trabajo (usuarios, peticiones, volumen de datos) sin degradar su rendimiento, ya sea aumentando recursos o rediseñando su arquitectura.

Tipos de escalabilidad

- Escalabilidad vertical (scale up): consiste en aumentar la capacidad de un mismo servidor (más CPU, más RAM, mejor disco). Es sencilla de aplicar pero tiene un límite físico y supone un punto único de fallo.
- Escalabilidad horizontal (scale out): consiste en añadir más servidores o nodos que trabajen en paralelo, distribuyendo la carga entre ellos. Es la estrategia preferida en sistemas web modernos por su mayor tolerancia a fallos.

Técnicas y componentes habituales

- Balanceadores de carga (load balancers): distribuyen las peticiones entrantes entre varios servidores.
- Replicación de bases de datos: réplicas de lectura para distribuir consultas y reducir la carga sobre el nodo principal.
- Particionamiento de datos (sharding): dividir los datos en fragmentos distribuidos entre distintos servidores.
- Servicios sin estado (stateless): diseñar los servicios de modo que no dependan de información almacenada localmente en el servidor, facilitando añadir o quitar nodos.
- Colas de mensajes: para desacoplar procesos y absorber picos de carga procesando tareas de forma asíncrona.

2. Patrones de caché en aplicaciones web

El caché almacena temporalmente resultados o datos de acceso frecuente para evitar recalcularlos o volver a consultarlos, reduciendo la latencia y la carga sobre los sistemas de backend.

Niveles de caché

- Caché de navegador (cliente): almacena recursos estáticos (imágenes, CSS, JS) localmente en el dispositivo del usuario.
- Caché en CDN (Content Delivery Network): distribuye copias de contenido estático en servidores geográficamente cercanos al usuario.
- Caché de aplicación: almacena en memoria (por ejemplo, con Redis o Memcached) resultados de operaciones costosas o consultas frecuentes.
- Caché de base de datos: mecanismos internos del motor de base de datos para acelerar consultas repetidas.

Estrategias de caché

- Cache-aside (lazy loading): la aplicación consulta primero el caché; si no encuentra el dato (cache miss), lo obtiene del origen y lo guarda en caché para futuras consultas.
- Write-through: los datos se escriben simultáneamente en el caché y en el almacenamiento persistente, manteniendo ambos sincronizados.
- Write-back (write-behind): los datos se escriben primero en el caché y, posteriormente, de forma asíncrona, en el almacenamiento persistente.
- Políticas de expiración (TTL - Time To Live) y de reemplazo (LRU, LFU) para gestionar qué datos permanecen en caché.

Consideraciones

- El caché mejora el rendimiento, pero introduce el problema de la consistencia de datos (invalidación de caché).
- Debe definirse claramente cuándo y cómo se invalida o actualiza cada dato cacheado.

3. Documentación y evaluación de arquitecturas web

Documentar y evaluar una arquitectura es fundamental para comunicarla a los distintos interesados (stakeholders), justificar decisiones de diseño y verificar que cumple con los requisitos de calidad esperados.

Documentación de arquitecturas

- Vistas arquitectónicas: representaciones del sistema desde distintas perspectivas (lógica, de despliegue, de procesos, de datos), como propone el modelo de vistas 4+1.
- Diagramas UML: diagramas de componentes, de despliegue, de secuencia, entre otros, para representar la estructura y el comportamiento del sistema.
- Registro de decisiones arquitectónicas (ADR - Architecture Decision Records): documentos breves que recogen qué decisión se tomó, por qué y qué alternativas se consideraron.

Evaluación de arquitecturas

- Atributos de calidad a evaluar: rendimiento, disponibilidad, seguridad, mantenibilidad, escalabilidad, usabilidad.
- Métodos de evaluación como ATAM (Architecture Tradeoff Analysis Method), que identifica los puntos de riesgo, sensibilidad y compromiso (trade-off) entre distintos atributos de calidad.
- Pruebas de carga y estrés para validar el comportamiento del sistema bajo condiciones reales o extremas de uso.
- Revisión por pares (peer review) y auditorías arquitectónicas periódicas para asegurar que la arquitectura evoluciona de forma coherente con los requisitos del negocio.

Referencias bibliográficas

1. Microsoft Learn / Azure Architecture Center. Cache-Aside Pattern. [https://learn.microsoft.com/en-us/previous-versions/msp-n-p/dn589799\(v=pandp.10\)](https://learn.microsoft.com/en-us/previous-versions/msp-n-p/dn589799(v=pandp.10))

2. Hazelcast. A Hitchhiker's Guide to Caching Patterns. <https://hazelcast.com/blog/a-hitchhikers-guide-to-caching-patterns/>
3. Software Engineering Institute, Carnegie Mellon University. The Architecture Tradeoff Analysis Method. <https://www.sei.cmu.edu/library/the-architecture-tradeoff-analysis-method-2>
4. Wikipedia. Architecture Tradeoff Analysis Method. https://en.wikipedia.org/wiki/Architecture_tradeoff_analysis_method